Why Your AI Replies Feel Unpredictable (And the Simple Fix That Works on Your Laptop)

Have you ever asked an AI the same question twice… and gotten two completely different answers?

Or switched from one AI model to another, only to find your perfectly working prompt suddenly broken?

You're not imagining it.

The way an AI responds can change dramatically based on tiny details: the wording of your prompt, the model you're using, or even random chance. This inconsistency makes it nearly impossible to build reliable tools or automate tasks.

But what if you could make AI replies **predictable**, **consistent**, and **ready to use in your own apps** — even on a regular laptop?

Good news: you can.
And it all comes down to one powerful idea: **Structured Output**.

In this article, I'll show you how to get clean, reliable responses from a small AI model running **right on your PC** — using just a few lines of Python. No PhD, no cloud costs, no magic required.

> 💡 **What You'll Need:**
> • A Windows, Mac, or Linux computer
> • Python installed
> • A PDF file to analyze
> • 5–10 minutes to set up

What Is Structured Output? (And Why It Changes Everything)

At its core, **structured output** means telling the AI:
"Don't just write whatever you want. Follow this exact format."

Instead of getting a messy paragraph like this:

> "Here's a quick summary of your document. It talks about China's environmental policies and long-term planning advantages. Interesting stuff!"

…you get something clean and organized like this:

```
{
  "summary": "China's centralized governance enables long-term planning, allowing aggressive renewable energy deployment and an ea
  "topics": "Environmental policy, long-term planning, energy transition, developing nations' leadership",
  "followup": [
    {
      "question": "How does China's political system support long-term climate goals?"
    },
    {
      "question": "Can democratic countries match China's pace in energy transition?"
    }
  ]
}
```

This is **JSON** — a universal data format that looks a lot like a dictionary in Python. Think of it as a **digital form** with labeled fields.

> 🔍 **What's JSON?**
> JSON (JavaScript Object Notation) is a lightweight way to store and exchange data. It's used everywhere in apps, websites, and APIs. For our purpose, it's the perfect way to get structured, machine-readable AI output.

How Do We Make AI Output Structured?

There are three main ways to guide an AI into giving structured replies. We'll only use the **easiest one**.

1. Schema-Based Output (Our Choice)

You define a "blueprint" for the response (like a form template), and the AI fills it out. The system checks that every field is present and correct.
✅ Easy, reliable, and perfect for beginners.

2. Grammar-Based Constraints (Advanced)

You write strict rules for how the output must be formatted (e.g., "must start with `{`, then a quote..."). The AI builds the response token by token, never breaking the rules.
🔧 Powerful, but complex — often used in local models.

3. Logit-Level Control (Expert Only)

You dive into the AI's internal mechanics and manually block invalid words during generation.
🔬 Maximum control, but way too technical for most people.

We'll go with **Method #1**: schema-based output. It's like filling out a form — simple, safe, and works great even on small models.

Let's Build a Simple AI Assistant (No Experience Needed)

Our goal:
Create a tool that:
1. Opens a PDF file
2. Extracts the text
3. Asks a local AI to return a **structured summary** (in JSON)
4. Lets us use that data in code

We'll use:
• **LFM2-1.2B**: A tiny but smart AI model that runs on your laptop
• **llama.cpp**: Software that runs the model and acts like an OpenAI API
• **Python**: To glue everything together

You don't need a GPU. A normal laptop is enough.

Step 1: Install the Tools

First, install the required Python packages:

```
pip install openai pydantic rich easygui pypdf tiktoken
```

These let us:
• Talk to the AI (`openai`)
• Define response formats (`pydantic`)
• Display results nicely (`rich`)
• Pick PDF files easily (`easygui`)
• Read PDFs (`pypdf`)
• Estimate text length (`tiktoken`)

> ❓ **What's a "token"?**
> In AI, a token is roughly a word or piece of a word. Models can only handle so many tokens at once — like a page limit. We'll check this to avoid errors.

Step 2: Run the AI Model Locally

We'll use **LFM2-1.2B**, a compact AI model designed to run on consumer devices.

Setup Steps:

1. Create a folder called `LFM2` on your computer.
2. Download the **llama.cpp server** for Windows:
   → llama-b6115-bin-win-cpu-x64.zip
   Extract the ZIP file into your `LFM2` folder.
3. Download the model file:
   → LFM2-1.2B-Q6_K.gguf
   Save it in the same folder.
4. Open Command Prompt in the folder and run:

```
llama-server.exe -m LFM2-1.2B-Q6_K.gguf -c 8192 -a FLM2
```

After some loading messages, you'll see:

```
Server is ready! OpenAI-compatible API is available at http://127.0.0.1:8080
```

🎉 Congratulations! You now have your own AI server running locally.

Step 3: Write the Python Code

Now, let's write the code that turns any PDF into structured insights.

Helper Functions

```
from openai import OpenAI
from pydantic import BaseModel, Field
from typing import List
import pypdf
import tiktoken
from easygui import fileopenbox
from rich import print as rprint

# Count how many tokens are in the text
def count_tokens(text):
    if not text:
        return 0
    encoding = tiktoken.get_encoding("cl100k_base")
    return len(encoding.encode(text))

# Convert PDF to plain text
def pdf_to_text(pdf_path):
    try:
        reader = pypdf.PdfReader(pdf_path)
        text = ""
        for page in reader.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
        cleaned = text.replace('\n', ' ').strip()
        rprint(f"[green]✅ Extracted {len(reader.pages)} pages[/green]")
        rprint(f"[blue]📝 Total tokens: {count_tokens(cleaned)}[/blue]")
        return cleaned
    except Exception as e:
```

```
        rprint(f"[red]⚠️ Error reading PDF: {e}[/red]")
        return None

# Open a file picker to select a PDF
def open_pdf():
    path = fileopenbox("Pick a PDF file", "*.pdf")
    return pdf_to_text(path)
```

Define the Output Structure

```
class Suggestion(BaseModel):
    question: str = Field(..., description="A follow-up question")

class Analysis(BaseModel):
    summary: str = Field(..., description="A concise summary of the document")
    topics: str = Field(..., description="Main themes discussed")
    followup: List[Suggestion] = Field(..., description="List of suggested questions")
```

Connect to the Local AI

```
client = OpenAI(
    base_url="http://localhost:8080/v1",
    api_key="not-required"
)
```

Send the Request

```
def generate_summary(text):
    system_prompt = "Analyze the document and return a JSON response with: summary, topics, and 2-3 follow-up questions."

    completion = client.beta.chat.completions.parse(
        model="local-model",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": text}
        ],
        response_format=Analysis
    )
    return completion.choices[0].message.parsed
```

Step 4: Run It!

```
# Step 1: Pick and read a PDF
context = open_pdf()
if not context:
    rprint("[red]❌ No text to process.[/red]")
else:
    # Step 2: Generate structured output
    result = generate_summary(context)

    # Step 3: Print the clean result
    rprint("[bold green]✨ Structured Output:[/bold green]")
    rprint(result)

    # You can now use the data reliably:
    rprint(f"\n[bold]Summary:[/bold] {result.summary}")
    rprint(f"[bold]First follow-up:[/bold] {result.followup[0].question}")
```

🧠 Beginner's Cheat Sheet

Goal: Get AI to reply in a fixed format (JSON) — every time.

Tools: Python + LFM2 + llama.cpp

Key Line: `response_format=Analysis` — forces the AI to follow your structure.

Use Cases: Summarization, data extraction, RAG, automation.

Pro Tip: Start with a short text to test the code before using large PDFs.

Why This Changes Everything

With structured output, you're no longer at the mercy of random AI replies.

You can now:
• Build chatbots that always return actionable data
• Create document processors that extract key info automatically
• Design apps where AI output feeds directly into databases or UIs

And the best part?
It works **on your laptop**, with **free tools**, using **open models**.

You don't need GPT-4 or Azure credits.
You don't need to be a machine learning engineer.

Just Python, a small model, and a clear structure.

Next: How to add Retrieval-Augmented Generation (RAG) to cite sources and quotes.